

#

#

#

#

```
random_state=int(rng.integers(1e6)))
```

```

samples = samples[(samples >= 1) & (samples <= max_val)]
shift = int(np.median(samples) - median)
samples = np.clip(samples - shift, 1, max_val)
return rng.choice(samples, size=n, replace=len(samples) < n).astype(float)

data_alta = simulate_nbinom(n_alta, med_alta, p25_alta, p75_alta, max_alta, rng)
data_obito = simulate_nbinom(n_obito, med_obt, p25_obt, p75_obt, max_obt, rng)
data_overall = np.concatenate([data_alta, data_obito]) # dados reais combinados

#

```

```

# 3. PALETA E LAYOUT
#

```

```

BG = "#FFFFFF"
PANEL = "#F6F8FA"
BORDER = "#D0D7DE"
TEXT = "#1F2328"
SUBTEXT = "#57606A"
COR_OV = "#7C6FC0" # roxo — Overall
COR_ALT = "#1D9E75" # verde — Discharge
COR_OBT = "#E07B39" # laranja — Death

# posições Y: Overall=2, Death=1, Discharge=0
POS = {"Overall": 2.0, "Death": 1.0, "Discharge": 0.0}

groups = [
    # (label, data, cor, n, med, ic_lo, ic_hi, p25, p75, mean, dp)
    ("Overall", data_overall, COR_OV, n_overall,
     med_overall, ic_lo_ov, ic_hi_ov, p25_overall, p75_overall, mean_overall, dp_overall),
    ("Discharge", data_alta, COR_ALT, n_alta,
     med_alta, ic_lo_alta, ic_hi_alta, p25_alta, p75_alta, mean_alta, dp_alta),
    ("Death", data_obito, COR_OBT, n_obito,
     med_obt, ic_lo_obt, ic_hi_obt, p25_obt, p75_obt, mean_obt, dp_obt),
]

#

```

```

# 4. FIGURA
#

```

```

fig, ax = plt.subplots(figsize=(12, 8), facecolor=BG)
ax.set_facecolor(PANEL)
for spine in ax.spines.values():
    spine.set_edgecolor(BORDER)
    spine.set_linewidth(0.9)

for label, data, cor, n, med, ic_lo, ic_hi, p25, p75, mean_, dp_ in groups:
    yc = POS[label]

    # — 1. KDE (meia-nuvem)

```

```

    kde = gaussian_kde(data, bw_method=0.35)
    x_grid = np.linspace(0, data.max() + 2, 300)
    kde_vals = kde(x_grid)
    kde_vals /= kde_vals.max()
    kde_scale = 0.36

```

```

ax.fill_between(x_grid,
                yc + 0.03,
                yc + 0.03 + kde_vals * kde_scale,
                color=cor, alpha=0.28, zorder=2)
ax.plot(x_grid,
        yc + 0.03 + kde_vals * kde_scale,
        color=cor, linewidth=1.4, alpha=0.75, zorder=3)

```

— 2. Boxplot

```

bw = 0.09
by = yc - 0.07

ax.add_patch(mpatches.FancyBboxPatch(
    (p25, by - bw), p75 - p25, 2 * bw,
    boxstyle="round,pad=0.5", linewidth=1.3,
    edgecolor=cor, facecolor=cor, alpha=0.18, zorder=4))
ax.add_patch(mpatches.FancyBboxPatch(
    (p25, by - bw), p75 - p25, 2 * bw,
    boxstyle="round,pad=0.5", linewidth=1.3,
    edgecolor=cor, facecolor="none", zorder=5))

iqr = p75 - p25
w_lo = max(data.min(), p25 - 1.5 * iqr)
w_hi = min(data.max(), p75 + 1.5 * iqr)
ax.plot([w_lo, p25], [by, by], color=cor, lw=1.3, zorder=4)
ax.plot([p75, w_hi], [by, by], color=cor, lw=1.3, zorder=4)
ax.plot([w_lo, w_lo], [by - bw * 0.5, by + bw * 0.5], color=cor, lw=1.3, zorder=4)
ax.plot([w_hi, w_hi], [by - bw * 0.5, by + bw * 0.5], color=cor, lw=1.3, zorder=4)

outliers = data[(data < w_lo) | (data > w_hi)]
if len(outliers):
    ax.scatter(outliers, np.full_like(outliers, by),
               s=16, color=cor, alpha=0.32, zorder=4, marker="o")

# mediana — diamante + IC
ax.plot(med, by, marker="D", markersize=9,
        color=cor, markerfacecolor=BG,
        markeredgecolor=cor, markeredgewidth=2.0, zorder=7)
ax.plot([ic_lo, ic_hi], [by, by],
        color=cor, lw=2.0, zorder=6, solid_capstyle="round")

```

— 3. Jitter

```

jitter_n = min(n, 250)
sample = rng.choice(data, size=jitter_n, replace=False)
jitter_y = rng.uniform(-0.05, 0.05, size=jitter_n) + yc - 0.22
ax.scatter(sample, jitter_y,
           s=10, color=cor, alpha=0.22, zorder=3, linewidths=0)

```

— 4. Anotações de texto

```

# Label e n (esquerda)
ax.text(-4.5, yc + 0.03 + kde_scale + 0.04,
        label, color=cor, fontsize=18, fontweight="bold",
        ha="right", va="bottom")
ax.text(-4.5, yc + 0.03 + kde_scale - 0.04,
        f'n = {n}', color=SUBTEXT, fontsize=15,
        ha="right", va="top")

```

```

# Median + CI (sem IQR — vai nas extremidades)
ax.text(ic_hi + 0.75, by + bw - 0.16,
       f'Median: {med:.0f} days\n95% CI: {ic_lo:.0f}–{ic_hi:.0f}',
       color=cor, fontsize=10, va="bottom", ha="left")

# Q1 à esquerda do boxplot
ax.text(w_lo - 0.9, by - 0.10,
       f'Q1={p25:.0f}',
       fontsize=10, color=cor, ha="right", va="center")

# Q3 à direita do boxplot
ax.text(w_hi + 0.9, by - 0.10,
       f'Q3={p75:.0f}',
       fontsize=10, color=cor, ha="left", va="center")

# — NOVO: Média ± DP (abaixo do boxplot) —————
ax.text(p25, by - bw - 0.215,
       f' $\bar{x} = \{mean\_:.1f\} \pm \{dp\_:.2f\} \text{ days}$ ',
       color=cor, fontsize=10, va="top", ha="left",
       style="italic")

#

```

```

# 5. DELTA MEDIANA — seta entre Death e Discharge
#

```

```

delta_md = med_obt - med_alta      # 5 - 3 = 2
x_arrow = max(max_alta, max_obt) * 0.51 # posição X da seta

by_disc = POS["Discharge"] - 0.08
by_death = POS["Death"] - 0.08

# linha vertical conectando as duas medianas
ax.annotate("",
            xy=(x_arrow, by_death),
            xytext=(x_arrow, by_disc),
            arrowprops=dict(
                arrowstyle="<->",
                color="#444444",
                lw=2.4,
                shrinkA=3, shrinkB=3,
            ),
            zorder=8)

# traços horizontais de ancoragem
for yy in [by_disc, by_death]:
    ax.plot([x_arrow - 0.6, x_arrow], [yy, yy],
            color="#444444", lw=2.0, zorder=7)

# texto do delta
ax.text(x_arrow + 0.8,
       (by_disc + by_death) / 2,
       f' $\Delta Md = \{delta\_md:.0f\} \text{ day}\{'s' \text{ if } delta\_md \neq 1 \text{ else } ''\}$ ',
       color="#333333", fontsize=10, fontweight="bold",
       va="center", ha="left")

#

```

```

# 6. ANOTAÇÃO DO MODELO BINOMIAL NEGATIVA

```

#

```
ax.text(0.98, 0.04,
        f"Negative Binomial Model\n"
        f" $\beta = \{\text{beta}:.4f\}$  (95% CI:  $\{\text{beta\_lo}:.3f\}$ – $\{\text{beta\_hi}:.3f\}$ )  $p = \{\text{p\_val}:.3f\}$ ",
        transform=ax.transAxes,
        color=TEXT, fontsize=12, ha="right", va="bottom",
        bbox=dict(boxstyle="round,pad=0.5", facecolor=BG,
                  edgecolor=BORDER, linewidth=0.9))
```

#

7. EIXOS, GRADE, LEGENDA, TÍTULO

#

```
ax.xaxis.grid(True, color=BORDER, linewidth=0.9, alpha=0.8, zorder=0)
ax.set_axisbelow(True)
ax.set_xlim(-4, max(max_alta, max_obt) * 0.58)
ax.set_ylim(-0.58, 2.72)
ax.set_yticks([])
ax.tick_params(axis="x", colors=SUBTEXT, labelsz=11)
ax.set_xlabel("Hospital length of stay (LOS)", fontsize=18,
              color=SUBTEXT, labelpad=8)
```

```
legend_elements = [
    mpatches.Patch(facecolor=COR_OV, edgecolor=COR_OV,
                   alpha=0.4, label="Overall"),
    mpatches.Patch(facecolor=COR_ALT, edgecolor=COR_ALT,
                   alpha=0.4, label="Discharge (0)"),
    mpatches.Patch(facecolor=COR_OBT, edgecolor=COR_OBT,
                   alpha=0.4, label="Death (1)"),
    Line2D([0], [0], marker="D", color="none",
           markerfacecolor=BG, markeredgecolor=TEXT,
           markeredgewidth=1.8, markersize=8,
           label="Median (95% CI)"),
    mpatches.Patch(facecolor="none", edgecolor=TEXT,
                   linewidth=1.2, label="IQR (P25–P75)"),
]
```

```
ax.legend(handles=legend_elements, fontsize=12, frameon=True,
          edgecolor=BORDER, facecolor=BG, labelcolor=TEXT,
          loc="upper right", framealpha=0.97,
          borderpad=0.5, handlelength=1.9)
```

```
##### RETIREI O TITULO E SUBTITULO #####
```

```
#ax.set_title("Distribution of Hospital Length of Stay According to Outcome",
#             fontsize=25, fontweight="bold", color=TEXT, pad=30)
```

```
#ax.text(0.5, 1.01,
#        "Discharge vs. Death | n = 703",
#        transform=ax.transAxes,
#        fontsize=15, color=SUBTEXT, ha="center", va="bottom")
```

```
plt.tight_layout(rect=[0, 0.02, 1, 1])
```

```
import os
```

```
OUTPUT_PNG = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                           "raincloud_dias_permanencia_v2.png")
```

```
plt.savefig(OUTPUT_PNG, dpi=180, bbox_inches="tight", facecolor=BG)
```

```
print(f" Salvo em: {OUTPUT_PNG}")
```

```
print(" Salvo em raincloud_dias_permanencia_v2.png")
```

```
plt.show()
```